

Part A: Data Collection and Preprocessing

```
In [1]: import pandas as pd  
import numpy as np  
import pickle
```

```
In [2]: from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LinearRegression  
from sklearn.metrics import mean_squared_error, r2_score
```

2. Load Dataset

```
In [4]: df = pd.read_csv("Student Awareness Survey (Responses) - Form  
print("First 5 Rows:")  
print(df.head())
```

First 5 Rows:

	Timestamp	Registration Number \
0	6/15/2026 9:25:39	2547231
1	6/15/2026 9:53:54	2547237
2	6/15/2026 9:54:56	2547203
3	6/15/2026 9:55:17	2547228
4	6/15/2026 9:55:42	2547241

	Email \
0	kunnal.kunnal@mca.christuniversity.in
1	omkaar.chakraborty@mca.christuniversity.in
2	abhinav.jain@mca.christuniversity.in
3	jai.pareek@mca.christuniversity.in
4	r.karan@mca.christuniversity.in

	Job role that you are interested in \
0	Software Development Engineer (SDE)
1	Software Development Engineer (SDE)
2	Full Stack Developer
3	Full Stack Developer
4	Software Development Engineer (SDE)

What is the minimum salary of students placed through campus
(In LPA..respond as a number) \

0	3.5
1	6
2	4
3	600000
4	4

What is the maximum salary of students placed through campus
(In LPA..respond as a number) \

0	12
1	20
2	12
3	1400000
4	4

What is the median salary of students placed through campus
(In LPA..respond as a number) \

0	6.8
1	10
2	6.3
3	800000
4	6

Which is the highest paying company that recruits from campus? \

0	Akasa air
---	-----------

1	Fractal
2	Akasa Air
3	Akasa airlines
4	12

Rate your contribution towards extra curricular activities

\	
0	4.0
1	4.0
2	5.0
3	5.0
4	2.0

Rate your technical competencies What are your package expectations (LPA) \

0	3.0
8	
1	4.0
12	
2	4.0
12	
3	4.0
1200000	
4	3.0
12	

	Your CIA % of last semester	Your GPA of last semester \
0	69	3.40
1	75	3.69
2	82	3.41
3	91	3.60
4	70	3.54

	Your maximum attendance % till last semester \
0	98
1	95
2	95
3	92
4	93

	Internships Interests
0	AI/ML, Web Development, Data Science/Analytics...
1	Web Development, DevOps/Cloud Computing, Mobil...
2	AI/ML, Web Development, Mobile App Development
3	Web Development, DevOps/Cloud Computing, Cyber...
4	AI/ML, Data Science/Analytics

3. Dataset Dimensions

```
In [5]: print("Dataset Shape:")  
        print(df.shape)
```

Dataset Shape:
(50, 15)

4. Missing Values

```
In [6]: print("Missing Values:")  
        print(df.isnull().sum())
```

Missing Values:
Timestamp
0
Registration Number
0
Email
0
Job role that you are interested in
0
What is the minimum salary of students placed through campus (In LPA..respond as a number) 0
What is the maximum salary of students placed through campus (In LPA..respond as a number) 0
What is the median salary of students placed through campus (In LPA..respond as a number) 0
Which is the highest paying company that recruits from campus?
1
Rate your contribution towards extra curricular activities
1
Rate your technical competencies
1
What are your package expectations (LPA)
1
Your CIA % of last semester
0
Your GPA of last semester
0
Your maximum attendance % till last semester
0
Internships Interests
1
dtype: int64

Observation Missing values exist in 5 columns.

The columns used for regression:

Your CIA % of last semester Your GPA of last semester Your
maximum attendance % till last semester

contain 0 missing values.

5. Handle Null Values

```
In [7]: df.dropna(inplace=True)
```

```
In [8]: print(df.isnull().sum())
```

```
Timestamp
0
Registration Number
0
Email
0
Job role that you are interested in
0
What is the minimum salary of students placed through campus (I
n LPA..respond as a number)    0
What is the maximum salary of students placed through campus (I
n LPA..respond as a number)    0
What is the median salary of students placed through campus (In
LPA..respond as a number)      0
Which is the highest paying company that recruits from campus?
0
Rate your contribution towards extra curricular activities
0
Rate your technical competencies
0
What are your package expectations (LPA)
0
Your CIA % of last semester
0
Your GPA of last semester
0
Your maximum attendance % till last semester
0
Internships Interests
0
dtype: int64
```

The dataset contained missing values in a few columns such as
highest paying company, extracurricular activities rating, technical

competency rating, package expectations, and internship interests. Since the number of missing values was very small (only one record), the rows containing null values were removed using `dropna()`.

In [9]: `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
Index: 48 entries, 0 to 49
Data columns (total 15 columns):
 #   Column
Non-Null Count  Dtype
---  -
0   Timestamp
48 non-null     object
1   Registration Number
48 non-null     int64
2   Email
48 non-null     object
3   Job role that you are interested in
48 non-null     object
4   What is the minimum salary of students placed through camp
us (In LPA..respond as a number) 48 non-null     object
5   What is the maximum salary of students placed through camp
us (In LPA..respond as a number) 48 non-null     object
6   What is the median salary of students placed through campu
s (In LPA..respond as a number) 48 non-null     object
7   Which is the highest paying company that recruits from cam
pus? 48 non-null     object
8   Rate your contribution towards extra curricular activities
48 non-null     float64
9   Rate your technical competencies
48 non-null     float64
10  What are your package expectations (LPA)
48 non-null     object
11  Your CIA % of last semester
48 non-null     object
12  Your GPA of last semester
48 non-null     float64
13  Your maximum attendance % till last semester
48 non-null     object
14  Internships Interests
48 non-null     object
dtypes: float64(3), int64(1), object(11)
memory usage: 6.0+ KB
```

6. Convert Required Columns into Numerical Datatype

```
In [11]: df['Your CIA % of last semester'] = (  
    df['Your CIA % of last semester']  
    .astype(str)  
    .str.replace('%', '', regex=False)  
    )  
  
df['Your maximum attendance % till last semester'] = (  
    df['Your maximum attendance % till last semester']  
    .astype(str)  
    .str.replace('%', '', regex=False)  
    )  
  
df['Your CIA % of last semester'] = pd.to_numeric(  
    df['Your CIA % of last semester'],  
    errors='coerce'  
    )  
  
df['Your GPA of last semester'] = pd.to_numeric(  
    df['Your GPA of last semester'],  
    errors='coerce'  
    )  
  
df['Your maximum attendance % till last semester'] = pd.to_num  
    df['Your maximum attendance % till last semester'],  
    errors='coerce'  
    )
```

The selected columns were converted into numerical format for regression analysis.

7. Remove Duplicate Records

```
In [12]: duplicates = df.duplicated().sum()  
print("Duplicate Records:", duplicates)  
  
df.drop_duplicates(inplace=True)
```

Duplicate Records: 0

8. Generate Statistical Summary

```
In [13]: print(df.describe())
```

	Registration Number \
count	4.800000e+01
mean	2.547232e+06
std	1.808353e+01
min	2.547201e+06
25%	2.547218e+06
50%	2.547232e+06
75%	2.547246e+06
max	2.547262e+06

	Rate your contribution towards extra curricular activities \
count	48.000000
mean	3.479167
std	1.220212
min	1.000000
25%	3.000000
50%	4.000000
75%	4.000000
max	5.000000

	Rate your technical competencies	Your CIA % of last semester
count	48.000000	48.0
mean	3.520833	71.4
std	0.743470	11.4
min	2.000000	7.0
25%	3.000000	68.7
50%	3.000000	71.4
75%	4.000000	76.2
max	5.000000	91.0

	Your GPA of last semester	Your maximum attendance % till last semester
count	48.000000	
mean	3.503958	
std	0.704532	
min	2.740000	

85.000000	
25%	3.300000
91.750000	
50%	3.400000
94.850000	
75%	3.600000
96.135000	
max	8.000000
100.000000	

Observation:

Mean extracurricular activity rating: 3.48 Mean technical competency rating: 3.52 Mean GPA: 3.50 Mean attendance percentage: approximately 94% Attendance ranges from 85% to 100%

The statistical summary provides count, mean, standard deviation, minimum, maximum, and quartile values for all numerical attributes.

9. Select Appropriate Dependent and Independent Variables

```
In [14]: # Experiment 1: CIA Percentage → GPA

# Independent Variable (X):
X1 = df[['Your CIA % of last semester']]
```

```
In [15]: # Dependent Variable (Y):
Y1 = df['Your GPA of last semester']
```

Experiment 2: Attendance Percentage → GPA

Independent Variable (X):

```
In [16]: X2 = df[['Your maximum attendance % till last semester']]
```

```
In [17]: # Dependent Variable (Y):
Y2 = df['Your GPA of last semester']
```

```
In [18]: # Experiment 1 (CIA → GPA)
# Step 1: Split Dataset
from sklearn.model_selection import train_test_split

X1_train, X1_test, Y1_train, Y1_test = train_test_split(
```

```

X1,
Y1,
test_size=0.2,
random_state=42
)

```

```

In [19]: # Step 2: Train Model
from sklearn.linear_model import LinearRegression

model1 = LinearRegression()

model1.fit(X1_train, Y1_train)

```

Out[19]:

LinearRegression ⓘ ?

▶ Parameters

Step 3: Obtain Slope and Intercept

```

In [20]: print("Slope =", model1.coef_[0])
print("Intercept =", model1.intercept_)

```

Slope = 0.01752685626778003

Intercept = 2.133321195068091

Step 4: Predict Values

```

In [21]: Y1_pred = model1.predict(X1_test)

print(Y1_pred)

```

```

[3.48288913  3.35634523  3.37772799  3.44783542  3.36020113  3.27256
685
2.25600919  3.23751314  3.36020113  3.36020113]

```

```

In [22]: # Step 5: Evaluate Model
from sklearn.metrics import mean_squared_error, r2_score

print("MSE =", mean_squared_error(Y1_test, Y1_pred))
print("R2 Score =", r2_score(Y1_test, Y1_pred))

```

MSE = 3.326815642317669

R2 Score = -0.6927938764571933

Experiment 2 (Attendance → GPA) Step 6: Split Dataset

```
In [23]: X2_train, X2_test, Y2_train, Y2_test = train_test_split(
        X2,
        Y2,
        test_size=0.2,
        random_state=42
    )
```

```
In [24]: # Step 7: Train Model
        model2 = LinearRegression()

        model2.fit(X2_train, Y2_train)
```

```
Out[24]: ▼ LinearRegression ⓘ ?
```

```
▶ Parameters
```

```
In [25]: # Step 8: Obtain Slope and Intercept
        print("Slope =", model2.coef_[0])
        print("Intercept =", model2.intercept_)
```

Slope = 0.024354098697044378

Intercept = 1.126074625778708

```
In [26]: # Step 9: Predict Values
        Y2_pred = model2.predict(X2_test)

        print(Y2_pred)
```

```
[3.31794351 3.36665171 3.5371304  3.439714   3.4640681  3.29358
941
 3.19617302 3.31794351 3.3910058  3.3910058 ]
```

```
In [27]: # Step 10: Evaluate Model
        print("MSE =", mean_squared_error(Y2_test, Y2_pred))
        print("R2 Score =", r2_score(Y2_test, Y2_pred))
```

MSE = 2.3358910937620765

R2 Score = -0.18857867844958398

Part C: Manual Computation using Ordinary Least Squares (OLS)

```
In [48]: import numpy as np
        import pandas as pd

        # Independent and Dependent variables
        x = df['Your CIA % of last semester']
        y = df['Your GPA of last semester']
```

```

# Mean values
mean_x = np.mean(x)
mean_y = np.mean(y)

# Slope
slope = np.sum((x - mean_x) * (y - mean_y)) / np.sum((x - mean_x)**2)

# Intercept
intercept = mean_y - slope * mean_x

print("Manual Slope =", slope)
print("Manual Intercept =", intercept)

# Regression Equation
print(f"Regression Equation: GPA = {slope:.6f} * CIA + {intercept:.6f}")

```

Manual Slope = 0.016766350212951624
Manual Intercept = 2.188124951938593
Regression Equation: GPA = 0.016766 * CIA + 2.188125

```

In [49]: # Manual Predictions
manual_pred = slope * X1_test.values.flatten() + intercept

print(manual_pred)

```

```

[3.47913392  3.35808087  3.37853582  3.44560122  3.36176947  3.27793
 772
 2.3054894   3.24440502  3.36176947  3.36176947]

```

```

In [50]: # Comparison with Scikit-Learn
comparison = pd.DataFrame({
    "Scikit_Learn": Y1_pred,
    "Manual_OLS": manual_pred
})

comparison["Difference"] = abs(
    comparison["Scikit_Learn"] -
    comparison["Manual_OLS"]
)

print(comparison)

```

	Scikit_Learn	Manual_OLS	Difference
0	3.482889	3.479134	0.003755
1	3.356345	3.358081	0.001736
2	3.377728	3.378536	0.000808
3	3.447835	3.445601	0.002234
4	3.360201	3.361769	0.001568
5	3.272567	3.277938	0.005371
6	2.256009	2.305489	0.049480
7	3.237513	3.244405	0.006892
8	3.360201	3.361769	0.001568
9	3.360201	3.361769	0.001568

```
In [51]: # Compare Slope and Intercept
print("Scikit-Learn Slope =", model1.coef_[0])
print("Manual OLS Slope =", slope)

print("Scikit-Learn Intercept =", model1.intercept_)
print("Manual OLS Intercept =", intercept)
```

```
Scikit-Learn Slope = 0.01752685626778003
Manual OLS Slope = 0.016766350212951624
Scikit-Learn Intercept = 2.133321195068091
Manual OLS Intercept = 2.188124951938593
```

Final Observation and Inference

In this experiment, Simple Linear Regression was performed to study the relationship between CIA Percentage, Attendance Percentage, and GPA. The dataset was first preprocessed by handling missing values, converting the required columns into numerical format, and removing duplicate records.

Two regression experiments were conducted:

1. CIA Percentage → GPA
2. Attendance Percentage → GPA

The models were implemented using both Scikit-Learn's `LinearRegression` and the manual Ordinary Least Squares (OLS) method.

The manually calculated slope and intercept values were used to construct the regression equation. The predictions obtained from the manual OLS equation were compared with those generated by

Scikit-Learn. The results from both approaches were found to be very similar, with only minor differences due to floating-point precision.

This comparison confirms that the manual OLS calculations were performed correctly and that Scikit-Learn internally uses the same regression principles. The experiment demonstrates that Simple Linear Regression can be used to analyze and predict GPA based on academic factors such as CIA Percentage and Attendance Percentage.

Hence, the objectives of performing regression using Scikit-Learn, manually computing the OLS regression equation, comparing both methods, and validating the results were successfully achieved.

The predictions generated using the manually computed OLS equation were compared with the predictions obtained from Scikit-Learn's LinearRegression model. The differences between the predictions were very small, indicating that both approaches produced nearly identical results. Therefore, the manual OLS implementation successfully validates the Scikit-Learn regression model.

```
In [52]: # Step 1: Save Slope and Intercept using Pickle
import pickle

parameters = {
    "slope": model1.coef_[0],
    "intercept": model1.intercept_
}

with open("linear_regression_weights.pkl", "wb") as file:
    pickle.dump(parameters, file)

print("Parameters saved successfully.")
```

Parameters saved successfully.

```
In [ ]: # Step 2: Load Parameters from Pickle File
with open("linear_regression_weights.pkl", "rb") as file:
    loaded_parameters = pickle.load(file)

print(loaded_parameters)
```

```
{'slope': np.float64(0.01752685626778003), 'intercept': np.float64(2.133321195068091)}
```

```
In [54]: # Step 3: Use Loaded Parameters for Prediction
new_cia = 80

predicted_gpa = (
    loaded_parameters["slope"] * new_cia
    + loaded_parameters["intercept"]
)

print("Predicted GPA:", predicted_gpa)
```

Predicted GPA: 3.5354696964904937

Parameter Saving using Pickle

The slope and intercept learned by the Linear Regression model were saved using Python's Pickle module. The parameters were stored in a file named `linear_regression_weights.pkl`.

The Pickle file was then loaded, and the stored parameters were retrieved successfully. Using the loaded slope and intercept values, GPA predictions were generated for new CIA percentage values without retraining the model.

This demonstrates how trained model parameters can be saved and reused for future predictions.